AIRHOCKEY ARENA

Jacob Martinez
Export-Type Project

Summary:

Airhockey Arena is an air hockey roguelike where the player can face procedurally generated levels, facing off against AI opponents as the each side of the rink becomes more chaotic and customized.

Big Ticket Items/Project Description:

1. Environment:

The first detail of this project is creating the air hockey environment. The playspace is defined by a continuous surface surrounded by four walls, a floor, and two goals carved into the opposite walls. The puck is a physics object that can glide along the floor. When the puck enters the goal a score updates. The last objects in the environment are mallets for either a player or an AI agent to control; colliding the puck and mallet apply force to the puck, this interaction is how the player or agent scores goals.

The environment can be modified by a list of objects that can be placed on the floor. These objects serve to either modify the physics of the puck or provide physical barriers to the playspace. These objects cannot be placed within a small radius of the goal and they cannot completely block all paths to the goal.

The list of planned objects include:

- Boost Pad: a small area which multiples the current velocity of the puck if it traverses through it.
- Slow Zone: a small area which slows the current velocity of the puck if it traverses through it, making it easier to hit.
- Wall: a rectangular obstacle can be placed at different angles to obstruct the pucks movement.
- Pop Bumper: a small mushroom shaped obstacle that launches the puck away from it on contact.

More objects may be added, but for the scope of the project I only want to focus on these four.

2. Environmental Procedural Generation:

To create more stark differences between rounds of play, the environment will be procedurally generated. A Perlin Noise Map will be assigned to the dimensions of the floor. All generations will be seed based and done before the level can be played, it cannot be controlled by the agent.

Its heightmap will correlate to the four different environment modifying objects listed in the previous section. Its implementation will look something like the following:

- No obstacle: 0.0 - 0.4
- Boost Pad: 0.41 - 0.5.5
- Slow Pad: 0.56 - 0.7
- Pop Bumper: 0.71 - 0.9
- Walls: 0.91 - 1.0

I also imagine the map to have a low scale so we see a variety of different objects within each stage, a medium amount of octaves and persistence to balance the detail and a low lacunarity to keep a smooth balance of objects.

This may not be the final implementation, this plan just represents the idea that a large portion of the map should be open to make the goal still accessible.

3. Agent Action Space:

The agent has two sets of raycats. One that will perceive the entire environment that reflects off of the side walls and are stopped by walls parallel to the goal, so that it can find paths to shoot, while another that is stopped by any obstacle that specifically looks and alerts the agent of the puck's position. The goal is to have the agent see how it is supposed to score and approach the puck by two different processes.

4. MLAgent Training Regimine:

The agents that will play are MLAgents that are similar to the agents in the pushblock assignment, using supervised behavior cloning at first before improving on itself during self-play.

I will establish checkpoints for the agent for both difficulty and ability to interact with each different type of obstacle, which will repeat the supervised to self-play cycle. I hope to match the different versions of the MLAgent to environments with their different respective “favored obstacle” to maps consisting of that type of obstacle to ensure that the agent has the ability to interact with those objects meaningfully.

Unity PlayTests:

- For the first PlayTest, I want to run an algorithm that pathfinders the map to see if there are available pathways to the goal and if the agent can see those pathways. This will be important to ensure that the agent is actually able to score.
- The Second Playtest, will have the mallets move throughout the open space on the board. If the mallet is blocked off from more than 20% of the open area it will fail the test. This is important to ensure that the player and agent are actually able to act in the open areas and that their action space is not overly limited by the generation.